

Technical Report For Granite Guardian

Ines Mouine - smjg43

Krishna Chauhan - mknf66

Rayane El-Mselmi -

Adam Fallon -

Ewan

Adel

Introduction

1.1 Project summary: what is Granite Guardian and what does it do?

For an average driver, the dashboard of a car can be a source of stress. When a "Check Engine" light flickers on, it rarely provides context, leaving the user caught between the fear of a breakdown and the confusion of technical jargon. **Granite Guardian** was born from the desire to bridge this gap, turning cold, complex vehicle data into a supportive and transparent conversation

Our system allows users to upload **OBD-II** CSV files. Once uploaded, the platform performs an analysis of the vehicle's vital signs (ex: engine RPM, coolant temperature or fuel Trims).

Instead of just flagging a numerical anomaly, our integration of the **IBM Granite 3.0 2B** model allows the system to speak to the user, explaining why a value is concerning and what steps should be taken next. The goal is to move from reactive repairs to proactive care and saving users money and providing peace of mind

1.2: Access and Technical Setup

The system was developed to run entirely on the user's local machine. This ensures that sensitive vehicle diagnostic logs (which could technically reveal driving patterns or locations) never leave the user's hardware.

- The frontend was developed using React 18 and **TypeScript** which provides a "chic," high-contrast dashboard designed for readability in various lighting conditions. It utilizes Lucide-React for intuitive iconography and is accessible at “http://localhost:5173”.
- The backend is powered by a FastAPI server that handles data validation and diagnostic logic, accessible at “http://localhost:8000”. It uses SQLAlchemy as an ORM to communicate with a local SQLite database
- Rather than relying on cloud based LLMs that require high-latency internet connections and monthly subscriptions, we used Ollama to serve the Granite3.02B model locally. This specific model was chosen because it was the best version (ex: **Granite 3.0 8B**, **Granite 3.0 2B**) that balances automotive diagnostic accuracy and the ability to run quickly and smoothly on customer hardware

About the required Environment for Launch:

To ensure the system operates correctly, you must activate the following environment:

- **Python 3.12+ Virtual Environment** containing dependencies like **fastapi**, **uvicorn** and **pandas**.
- The **Ollama** application must be running in the system tray to handle AI requests.
- And **Node.js Environment**: To serve the React frontend via Vite

1.3: Status of Behavioural Requirements

ID	Requirement	State	Justification /Commentary
BR1	CSV File Ingestion	Completed	handling and validation of <code>KNOWN_SENSORS</code> data columns
BR2	Anomaly detection	Completed	automatic real-time comparison against safety thresholds defined in <code>config.py</code>
BR3	Granite AI Chatbot	Completed	local integration via Ollama for natural language explanations.
BR4	Dashboard	Completed	graphical representation of critical sensor data for user diagnostics
BR5	Alerting System	Completed	logging, filtering and acknowledgment functionality for security and safety alerts

Table 1: Status of Behavioral Requirements

Table 1 provides an overview of the system’s core functionalities as defined in the Requirements Specification. It tracks the successful implementation of each feature, ensuring alignment between the initial user stories and the final software deliverable